

Boggle Solver Code Review Summary Report

Amaia J. Murat

College of Engineering and Architecture, Howard University

CSCI-375-01: Software Engineering

Dr. Legand Burge

3 October 2026

Overview

This assignment simulates the maintenance phase of the Software Development Life Cycle (SDLC). During maintenance, developers review existing software, identify defects or areas for improvement, and make changes to improve its reliability, efficiency, and maintainability. My code review followed this process, as I developed and reviewed my Boggle Solver in the Amaia-Murat subfolder of the class repository, using a separate branch to isolate my changes from the main branch. Initially, I implemented my solver using a Trie to store a dictionary of valid words because the structure allows the program to determine whether a sequence of letters could form part of a valid word. I also used recursive backtracking to consider adjacent tiles, while keeping track of which cubes had already been used to prevent cube reuse within the same word. Lastly, I classified “Qu”, “St”, and “Ie” tiles as two-letter tiles, while ensuring that valid words contained at least three letters. My initial approach was to create a functional solution that correctly searches for different paths on the board, while planning to utilize my code review to identify room to improve efficiency and validation.

Feedback I Received

Upon reviewing my code with a peer, Ohene Osei (ohene-gif), I was advised to explicitly validate input types, as relying on a bare accept can hide unexpected programming errors by returning empty lists. Before my code review, I utilized my self.solutions list to check for duplicate words, but later initialized a set in its place, as checking through a list can become slow as more words are found. Likewise, Ohene suggested I create a reusable matrix for visited words before the starting-position loops, since findWords() already unmarks each tile while backtracking.

Feedback I Provided

While reviewing Ohene's initial Boggle Solver program, I recognized that he correctly used a Trie to efficiently identify valid Boggle words, enforced the three-letter minimum, and included diagonal movement in the traversal according to the assignment requirements. I suggested simplifying the duplicate solution variables and replacing magic numbers with named constants to improve readability and maintainability. I also identified vague function names and recommended using more descriptive names that clearly communicate each method's purpose, such as renaming `dfs` to `search_from_tile`. These changes would make the code easier to understand, modify, and maintain while preserving the existing algorithm.

Improvements I Implemented

During my code review, I made several changes to improve the efficiency, reliability, and maintainability of my Boggle Solver. Previously, I used the `self.solution` list to check for duplicate words, then implemented a separate set to track found words, since using a set allows the program to perform membership checks more efficiently while still maintaining the solution list for the final output. I also changed the visited matrix so that it is created once before the search begins and reused during recursive backtracking, rather than creating a new matrix for each starting position. Additionally, I added explicit input validation to check that the grid, dictionary, rows, and tiles contain the expected data types and dimensions. Finally, I removed broad exception handling and replaced it with explicit validation so that unexpected errors are not hidden. These changes were made to preserve the original Trie and recursive backtracking approach while making the program more efficient, predictable, and easier to maintain.

Static Analysis Results

After using static analysis to identify style and formatting issues in my Boggle Solver, I corrected issues such as inconsistent indentation, unnecessary whitespace between functions, and insufficient spacing in inline comments. I addressed these issues by standardizing indentation and comment formatting and removing trailing whitespace in alignment with the PEP 8 style guide. I also corrected similar style issues in my test file, without changing the functionality of the Boggle Solver. Through using the linter, I learned that consistency is important for maintaining code because it makes the program easier for peers and future developers to read, understand, and modify.

Regression Testing

To conduct regression testing, I added my 25 test cases to the existing test suite, bringing the total to 39 tests. These tests checked basic word finding, different grid and dictionary capitalization, duplicate and multiple words, `setGrid()` and `setDictionary()`, special Qu, St, and Ie tiles, invalid raw tiles, adjacent movement, cube reuse, minimum word length, and empty or small grids. All 25 of my test cases passed, along with the professor's Codio test suite, confirming that my code changes did not break the solver's existing functionality.

Reflection

Through this code review, I became more familiar with the purpose of branches in GitHub and learned how to create a pull request and conduct a code review. I also learned how to be an effective code reviewer by evaluating code for correctness and edge cases, algorithm efficiency, readability, maintainability, and overall code clarity. Reviewing code that was not written in my personal coding style also gave me practice in reading, understanding, and evaluating someone else's approach rather than judging it based on how I would have written it.

I learned to make my suggestions constructive by framing them as questions and assuming competence in each comment, which allowed me to focus on improving the code rather than criticizing the developer. Additionally, I learned the importance of considering Big-O notation when evaluating algorithm efficiency and how the growth of data can influence which data structure is most appropriate. For example, using a set for duplicate-word checking can become more beneficial as the amount of data increases. I also learned that explicitly validating inputs can help identify invalid data and prevent programming errors from causing undesired behavior in the future. Overall, this experience strengthened my ability to write, review, and communicate about code while giving me practical experience with collaborative development practices that will be valuable in a professional software engineering environment after graduation.